



Appendix

1. Data Exploration and Cleaning

Activity 1.01: Exploring the Remaining Financial Features in the Dataset

Solution:

Before beginning, set up your environment and load in the cleaned dataset as follows:

```
import pandas as pd
import matplotlib.pyplot as plt #import plotting
package
#render plotting automatically
%matplotlib inline
import matplotlib as mpl #additional plotting
functionality
mpl.rcParams['figure.dpi'] = 400 #high resolution
figures
mpl.rcParams['font.size'] = 4 #font size for figures
from scipy import stats
import numpy as np
df =
pd.read_csv('../Data/Chapter_1_cleaned_data.csv')
```

1. Create lists of feature names for the remaining financial features.

These fall into two groups, so we will make lists of feature names as before, to facilitate analyzing them together. You can do this with the following code:

```
bill_feats = ['BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3',
              \
              'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6']
pay_amt_feats = ['PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3',
                 \
                 'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6']
```

2. Use `.describe()` to examine statistical summaries of the bill amount features. Reflect on what you see. Does it make sense?

Use the following code to view the summary:

```
df[bill_feats].describe()
```

The output should appear as follows:

```
df[bill_feats].describe()
```

	BILL_AMT1	BILL_AMT2	BILL_AMT3	BILL_AMT4	BILL_AMT5	BILL_AMT6
count	26664.000000	26664.000000	26664.000000	26664.000000	26664.000000	26664.000000
mean	51405.730723	49300.001500	47026.340047	43338.894539	40338.136701	38889.872337
std	73633.687106	70934.549534	68705.359524	64275.250740	60705.944083	59432.541657
min	-165580.000000	-69777.000000	-157264.000000	-170000.000000	-81334.000000	-339603.000000
25%	3580.000000	2999.750000	2627.250000	2341.750000	1745.000000	1256.000000
50%	22361.000000	21150.000000	20079.500000	19037.000000	18066.000000	17005.000000
75%	67649.750000	64395.500000	60360.000000	54727.500000	50290.500000	49253.750000
max	746814.000000	671563.000000	855086.000000	706864.000000	823540.000000	699944.000000

Figure 1.47: Statistical description of bill amounts for the past 6 months

We see that the average monthly bill is roughly 40,000 to 50,000 NT dollars. You are encouraged to examine the conversion rate to your local currency. For example, 1 US dollar $\approx$ 30 NT dollars. Do the conversion and ask yourself, is this a reasonable monthly payment? We should also confirm this with the client, but it seems reasonable.

We also notice there are some negative bill amounts. This seems reasonable because of the possible overpayment of the previous month's bill, perhaps in anticipation of a purchase that would show up on the current month's bill. A scenario like this would leave that

account with a negative balance, in the sense of a credit to the account holder.

3. Visualize the bill amount features using a 2 by 3 grid of histogram plots using the following code:

```
df[bill_feats].hist(bins=20, layout=(2,3))
```

The graph should look like this:

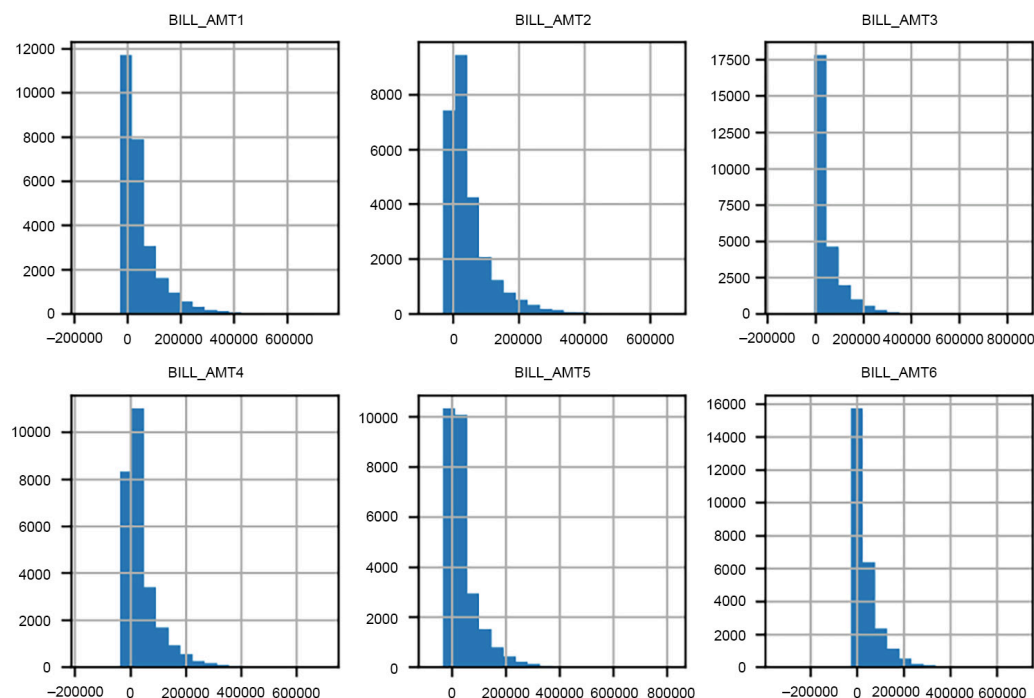


Figure 1.48: Histograms of bill amounts

The histogram plots in *Figure 1.48* make sense in several respects. Most accounts have relatively small bills. There is a steady decrease in the number of accounts as the amount of the bill increases. It also appears that the distribution of billed amounts is roughly similar month to month, so we don't notice any data inconsistency issues as we did with the payment status features. This feature appears to pass our data quality inspection. Now, we'll move on to the final set of features.

4. Use the `.describe()` method to obtain a summary of the payment amount features using the following code:

```
df[pay_amt_feats].describe()
```

The output should appear thus:

```
df[pay_amt_feats].describe()
```

	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6
count	26664.000000	2.666400e+04	26664.000000	26664.000000	26664.000000	26664.000000
mean	5704.085771	5.881110e+03	5259.514964	4887.048717	4843.729973	5257.843047
std	16699.398632	2.121431e+04	17265.439561	15956.349371	15311.721795	17635.468185
min	0.000000	0.000000e+00	0.000000	0.000000	0.000000	0.000000
25%	1000.000000	8.020000e+02	390.000000	294.750000	242.750000	111.000000
50%	2114.500000	2.007000e+03	1822.000000	1500.000000	1500.000000	1500.000000
75%	5027.000000	5.000000e+03	4556.250000	4050.500000	4082.750000	4015.000000
max	873552.000000	1.227082e+06	889043.000000	621000.000000	426529.000000	528666.000000

Figure 1.49: Statistical description of bill payment amounts for the past 6 months

The average payment amounts are about an order of magnitude (power of 10) lower than the average bill amounts we summarized earlier in the activity. This means that the "average case" is an account that is not paying off its entire balance from month to month. This makes sense in light of our exploration of the **PAY_1** feature, for which the most prevalent value was 0 (the account made at least the minimum payment but did not pay off the whole balance). There are no negative payments, which also seems right.

5. Plot a histogram of the bill payment features similar to the bill amount features, but also apply some rotation to the *x-axis* labels with the **xrot** keyword argument so that they don't overlap. Use the **xrot=<angle>** keyword argument to rotate the *x-axis* labels by a given angle in degrees using the following code:

```
df[pay_amt_feats].hist(layout=(2,3), xrot=30)
```

In our case, we found that 30 degrees of rotation worked well. The plot should look like this:

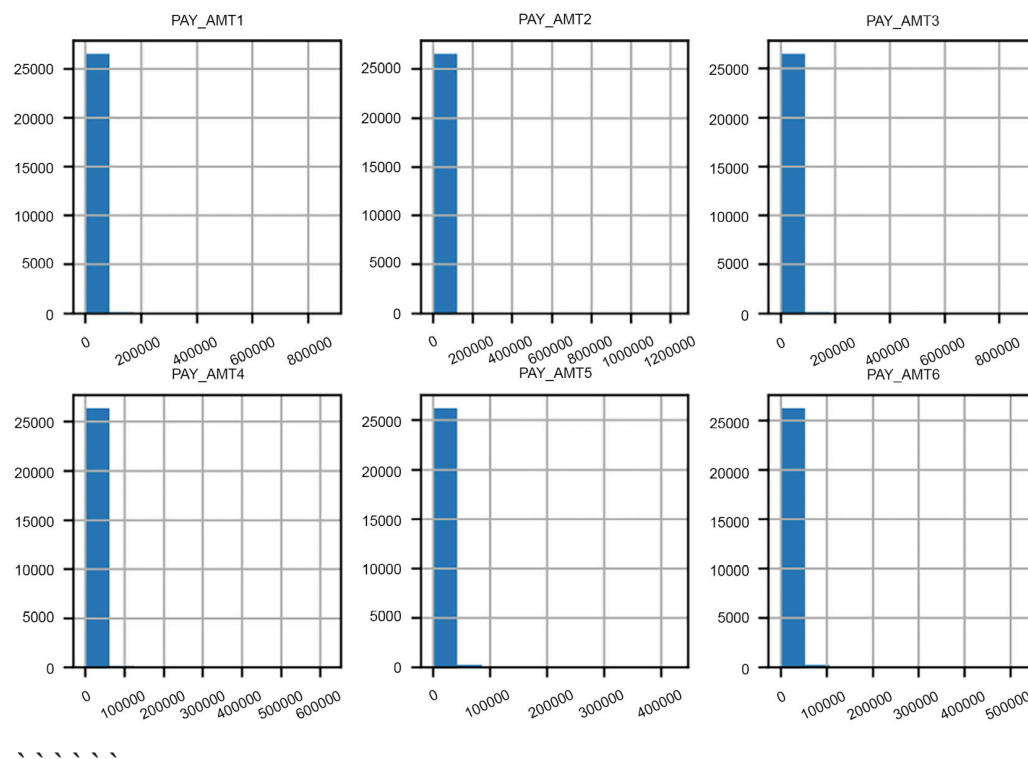


Figure 1.50: Histograms of raw payment amount data

A quick glance at this figure indicates that this is not a very informative graphic; there is only one bin in most of the histograms that is of a noticeable height. This is not an effective way to visualize this data. It appears that the monthly payment amounts are mainly in a bin that includes 0. How many are in fact 0?

6. Use a Boolean mask to see how much of the payment amount data is exactly equal to 0 using the following code: Do this with the following code:

```
pay_zero_mask = df[pay_amt_feats] == 0
pay_zero_mask.sum()
```

The output should look like this:

```
PAY_AMT1    4656
PAY_AMT2    4833
PAY_AMT3    5293
PAY_AMT4    5697
PAY_AMT5    5981
PAY_AMT6    6373
dtype: int64
```

Figure 1.51: Counts of bill payments equal to 0

Does this data make sense given the histogram in the previous step?

The first line here creates a new DataFrame called **pay_zero_mask**, which is a DataFrame of **True** and **False** values according to whether the payment amount is equal to 0. The second line takes the column sums of this DataFrame, interpreting **True** as 1 and **False** as 0, so the column sums indicate how many accounts have a value of 0 for each feature.

We see that a substantial portion, roughly around 20-25% of accounts, have a bill payment equal to 0 in any given month.

However, most bill payments are above 0. So, why can't we see them in the histogram? This is due to the **range** of values for bill payments relative to the values of the majority of the bill payments. In the statistical summary, we can see that the maximum bill payment in a month is typically 2 orders of magnitude (100 times) larger than the average bill payment. It seems likely there are only a small number of these very large bill payments. But, because of the way the histogram is created, using equal-sized bins, nearly all the data is lumped into the smallest bin, and the larger bins are nearly invisible because they have so few accounts. We need a strategy to effectively visualize this data.

7. Ignoring the payments of 0 using the mask you created in the previous step, use pandas' **.apply()** and NumPy's **np.log10()** method to plot histograms of logarithmic transformations of the non-zero payments. You can use **.apply()** to apply any function, including **log10**, to all the elements of a DataFrame. Use the following code for this:

```
df[pay_amt_feats][~pay_zero_mask].apply(np.log10)\
                                   .hist(layout=(2,3))
```

This is a relatively advanced use of pandas, so don't worry if you couldn't figure it out by yourself. However, it's good to start to get an impression of how you can do a lot in pandas with relatively little code.

The output should be as follows:

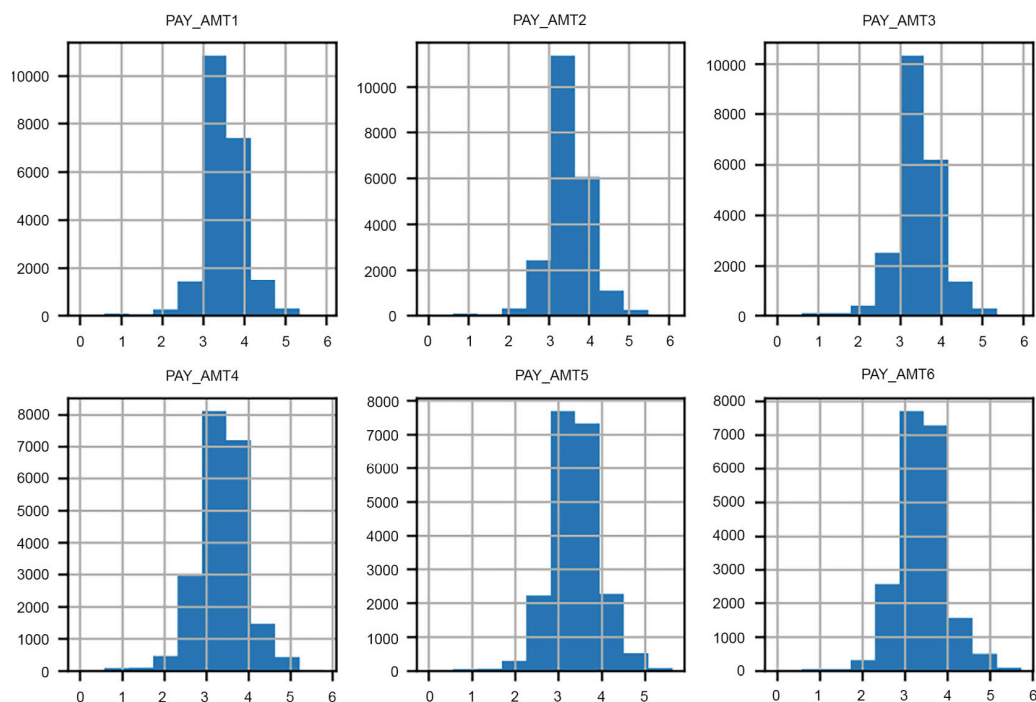


Figure 1.52: Base-10 logs of non-zero bill payment amounts

While we could have tried to create variable-width bins for better visualization of the payment amounts, a more convenient approach that is often used to visualize, and sometimes even model, data that has a few values on a much different scale than most of the values is a logarithmic transformation, or **log transform**. We used a base-10 log transform. Roughly speaking, this transform tells us the number of zeros in a value. In other words, a balance of at least 1 million dollars, but less than 10 million, would have a log transform of at least 6 but less than 7, because $10^6 = 1,000,000$ (and conversely $\log_{10}(1,000,000) = 6$) while $10^7 = 10,000,000$.

To apply this transformation to our data, first, we needed to mask out the zero payments, because $\log_{10}(0)$ is undefined (another common approach in this case is to add a very small number to all values, such as 0.01, so there are no zeros). We did this with the Python logical **not** operator `~` and the zero mask we created already. Then we used the pandas `.apply()` method, which applies any function we like to the data we have selected. In this case, we wished to apply a base-10 log-

rithm, calculated by `np.log10`. Finally, we made histograms of these values.

The result is a more effective data visualization: the values are spread in a more informative way across the histogram bins. We can see that the most commonly occurring bill payments are in the range of thousands ($\log_{10}(1,000) = 3$), which matches what we observed for the mean bill payment in the statistical summary. There are some pretty small bill payments, and also a few pretty large ones. Overall, the distribution of bill payments appears pretty consistent from month to month, so we don't see any potential issues with this data.

2. Introduction to Scikit-Learn and Model Evaluation

Activity 2.01: Performing Logistic Regression with a New Feature and Creating a Precision-Recall Curve

Solution:

1. Use scikit-learn's `train_test_split` to make a new set of training and test data. This time, instead of `EDUCATION`, use `LIMIT_BAL`, the account's credit limit, as the feature.

Execute the following code to do this:

```
X_train_2, X_test_2, y_train_2, y_test_2 =
train_test_split\
(df['LIMIT_
BAL']\
.values\
.reshape(-
1,1),\
df['defaul
t']\
```

```

        'paymen
t next'\
        'month'
    ].values,\
        test_size=
    0.2,\
        random_sta
te=24))

```

Notice here we create new training and test splits, with new variable names.

2. Train a logistic regression model using the training data from your split.

The following code does this:

```
example_lr.fit(X_train_2, y_train_2)
```

You can reuse the same model object you used earlier, **example_lr**, if you're running the whole chapter in a single notebook. You can **re-train** this object to learn the relationship between this new feature and the response. You could even try a different train/test split, if you wanted to, without creating a new model object. The existing model object has been updated **in-place** in these scenarios.

3. Create the array of predicted probabilities for the test data.

Here is the code for this step:

```

y_test_2_pred_proba =
example_lr.predict_proba(X_test_2)

```

4. Calculate the ROC AUC using the predicted probabilities and the true labels of the test data. Compare this to the ROC AUC from using the **EDUCATION** feature.

Run this code for this step:

```

metrics.roc_auc_score(y_test_2,
y_test_2_pred_proba[:,1])

```

The output is as follows:

```
0.6201990844642832
```

Notice that we index the predicted probabilities array in order to get the predicted probability of the positive class from the second

column. How does this compare to the ROC AUC from the **EDUCATION** logistic regression? The AUC is higher. This may be because now we are using a feature that has something to do with an account's financial status (credit limit), to predict something else related to the account's financial status (whether or not it will default), instead of using something less directly related to finances.

5. Plot the ROC curve.

Here is the code to do this; it's similar to the code we used in the previous exercise:

```
fpr_2, tpr_2, thresholds_2 = metrics.roc_curve\
                                (y_test_2, \
                                 y_test_2_pred_proba[:,1]
                                ])
plt.plot(fpr_2, tpr_2, '*-')
plt.plot([0, 1], [0, 1], 'r--')
plt.legend(['Logistic regression', 'Random chance'])
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title('ROC curve for logistic regression with '\
          'LIMIT_BAL feature')
```

The plot should appear as follows:

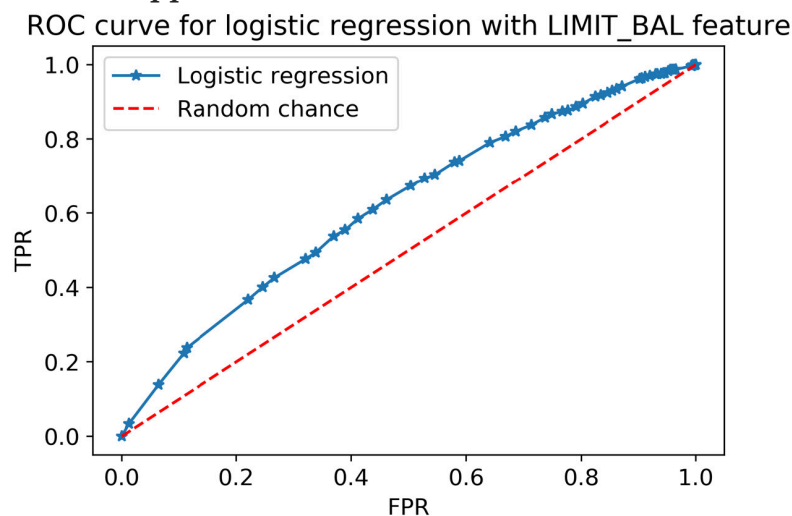


Figure 2.30: ROC curve for the LIMIT_BAL logistic regression

This looks a little closer to an ROC curve that we'd like to see: it's a bit further from the random chance line than the model using only **EDUCATION**. Also notice that the variation in pairs of true and false

positive rates is a little smoother over the range of thresholds, reflective of the larger number of distinct values of the **LIMIT_BAL** feature.

6. Calculate the data for the precision-recall curve on the test data using scikit-learn's functionality.

Precision is often considered in tandem with recall. We can use **precision_recall_curve** in **sklearn.metrics** to automatically vary the threshold and calculate pairs of precision and recall values at each threshold value. Here is the code to retrieve these values, which is similar to **roc_curve**:

```
precision, recall, thresh_3 =  
    metrics.precision_recall_curve\  
        (y_test_2,\  
         y_test_2_pred_proba[:,  
         1])
```

7. Plot the precision-recall curve using matplotlib: we can do this with the following code.

Note that we put recall on the **x**-axis and precision on the **y**-axis, and we set the axes' limits to the range [0, 1]:

```
plt.plot(recall, precision, '-x')  
plt.xlabel('Recall')  
plt.ylabel('Precision')  
plt.title('Precision and recall for the logistic\  
         'regression 'with LIMIT_BAL')  
plt.xlim([0, 1])  
plt.ylim([0, 1])
```

Precision and recall for the logistic regression with LIMIT_BAL

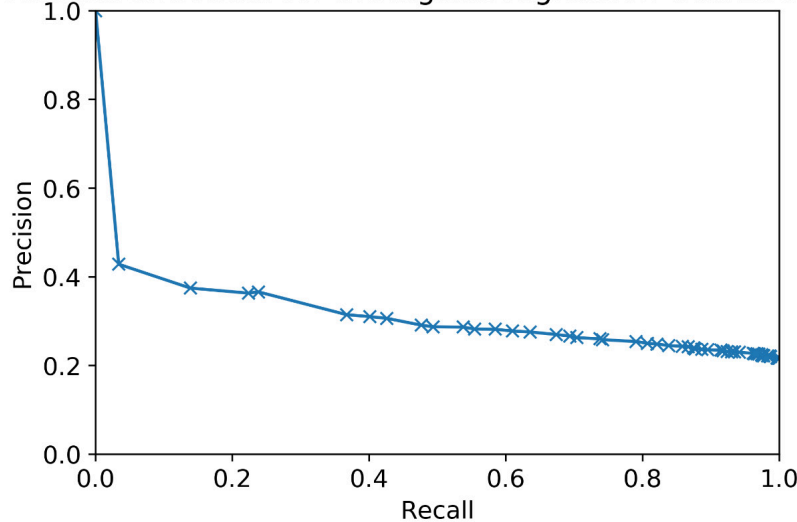


Figure 2.31: Plot of the precision-recall curve

8. Use scikit-learn to calculate the area under the precision-recall curve.

Here is the code for this:

```
metrics.auc(recall, precision)
```

You will obtain the following output:

```
0.31566964427378624
```

We saw that the precision-recall curve shows that precision is generally fairly low for this model; for nearly all of the range of thresholds, the precision, or portion of positive classifications that are correct, is less than half. We can calculate the area under the precision-recall curve as a way to compare this classifier with other models or feature sets we may consider.

Scikit-learn offers functionality for calculating an AUC for any set of **x-y** data, using the trapezoid rule, which you may recall from calculus: **metrics.auc**. We used this functionality to get the area under the precision-recall curve.

9. Now recalculate the ROC AUC, except this time do it for the training data. How is this different, conceptually and quantitatively, from your earlier calculation?

First, we need to calculate predicted probabilities using the training data, as opposed to the test data. Then we can calculate the ROC AUC using the training data labels. Here is the code:

```
y_train_2_pred_proba =  
example_lr.predict_proba(X_train_2)  
metrics.roc_auc_score(y_train_2,  
y_train_2_pred_proba[:,1])
```

You should obtain the following output:

```
0.6182918113358344
```

Quantitatively, we can see that this AUC is not all that different from the test data ROC AUC we calculated earlier. Both are about 0.62. Conceptually, what is the difference? When we calculate this metric on the training data, we are measuring the model's skill in predicting the same data that "taught" the model how to make predictions. We are seeing *how well the model fits the data*. On the other hand, test data metrics indicate performance on out-of-sample data the model hasn't "seen" before. If there was much of a difference in these scores, which usually would come in the form of a higher training score than the test score, it would indicate that while the model fits the data well, the trained model does not generalize well to new, unseen data.

In this case, the training and test scores are similar, meaning the model does about as well on out-of-sample data as it does on the same data used in model training. We will learn more about the insights we can gain by comparing training and test scores in *Chapter 4, The Bias-Variance Trade-Off*.

3. Details of Logistic Regression and Feature Exploration

Activity 3.01: Fitting a Logistic Regression Model and Directly Using the Coefficients

Solution:

The first few steps are similar to things we've done in previous activities:

1. Create a train/test split (80/20) with **PAY_1** and **LIMIT_BAL** as features:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    df[['PAY_1', 'LIMIT_BAL']].values,
    df['default payment next month'].values,
    test_size=0.2, random_state=24)
```

2. Import **LogisticRegression**, with the default options, but set the solver to '**liblinear**':

```
from sklearn.linear_model import LogisticRegression
lr_model = LogisticRegression(solver='liblinear')
```

3. Train on the training data and obtain predicted classes, as well as class probabilities, using the test data:

```
lr_model.fit(X_train, y_train)
y_pred = lr_model.predict(X_test)
y_pred_proba = lr_model.predict_proba(X_test)
```

4. Pull out the coefficients and intercept from the trained model and manually calculate predicted probabilities. You'll need to add a column of ones to your features, to multiply by the intercept. First, let's create the array of features, with a column of ones added, using horizontal stacking:

```
ones_and_features = np.hstack\
    ([np.ones((X_test.shape[0],1)),
    X_test])
```

Now we need the intercept and coefficients, which we reshape and concatenate from scikit-learn output:

```
intercept_and_coefs = np.concatenate\
    ([lr_model.intercept_.reshape(1,1), \
    lr_model.coef_], axis=1)
```

To repeatedly multiply the intercept and coefficients by all the rows of **ones_and_features**, and take the sum of each row (that is,

find the linear combination), you could write this all out using multiplication and addition. However, it's much faster to use the dot product:

```
X_lin_comb = np.dot(intercept_and_coefs,\n                    np.transpose(ones_and_features))
```

Now **X_lin_comb** has the argument we need to pass to the sigmoid function we defined, in order to calculate predicted probabilities:

```
y_pred_proba_manual = sigmoid(X_lin_comb)
```

5. Using a threshold of **0.5**, manually calculate predicted classes. Compare this to the class predictions outputted by scikit-learn. The manually predicted probabilities, **y_pred_proba_manual**, should be the same as **y_pred_proba**; we'll check that momentarily. First, manually predict the classes with the threshold:

```
y_pred_manual = y_pred_proba_manual >= 0.5
```

This array will have a different shape than **y_pred**, but it should contain the same values. We can check whether all the elements of two arrays are equal like this:

```
np.array_equal(y_pred.reshape(1,-1), y_pred_manual)
```

This should return a logical **True** if the arrays are equal.

6. Calculate ROC AUC using both scikit-learn's predicted probabilities and your manually predicted probabilities, and compare them. First, import the following:

```
from sklearn.metrics import roc_auc_score
```

Then, calculate this metric on both versions, taking care to access the correct column, or reshape as necessary:

```
roc_auc_score(y_test, y_pred_proba_manual.reshape(y_pred_proba_manual.shape[1],))\n0.627207450280691
```

```
roc_auc_score(y_test, y_pred_proba[:,1])\n0.627207450280691
```

Figure 3.37: Calculating the ROC AUC from predicted probabilities

The AUCs are, in fact, the same. What have we done here? We've confirmed that all we really need from this fitted scikit-learn model is three numbers: the intercept and the two coefficients. Once we have

these, we could create model predictions using a few lines of code, with mathematical functions, that are equivalent to the predictions directly made from scikit-learn.

This is good to confirm your understanding, but otherwise, why would you ever want to do this? We'll talk about **model deployment** in the final chapter. However, depending on your circumstances, you may be in a situation where you don't have access to Python in the environment where new features will need to be input into the model for prediction. For example, you may need to make predictions entirely in SQL. While this is a limitation in general, with logistic regression you can use mathematical functions that are available in SQL to re-create the logistic regression prediction, only needing to copy and paste the intercept and coefficients somewhere in your SQL code. The dot product may not be available, but you can use multiplication and addition to accomplish the same purpose.

Now, what about the results themselves? What we've seen here is that we can slightly boost model performance above our previous efforts: using just **LIMIT_BAL** as a feature in the previous chapter's activity, the ROC AUC was a bit less at 0.62, instead of 0.63 here. In the next chapter, we'll learn advanced techniques with logistic regression that we can use to further improve performance.

4. The Bias-Variance Trade-Off

Activity 4.01: Cross-Validation and Feature Engineering with the Case Study Data

Solution:

1. Select out the features from the DataFrame of the case study data. You can use the list of feature names that we've already created in this chapter, but be sure not to include the response variable,

which would be a very good (but entirely inappropriate) feature:

```
features = features_response[:-1]
X = df[features].values
```

2. Make a training/test split using a random seed of 24:

```
X_train, X_test, y_train, y_test = \
train_test_split(X, df['default payment next
month'].values,
                 test_size=0.2, random_state=24)
```

We'll use this going forward and reserve this test data as the unseen test set. By specifying the random seed, we can easily create separate notebooks with other modeling approaches using the same training data.

3. Instantiate **MinMaxScaler** to scale the data, as shown in the following code:

```
from sklearn.preprocessing import MinMaxScaler
min_max_sc = MinMaxScaler()
```

4. Instantiate a logistic regression model with the **saga** solver, L1 penalty, and set **max_iter** to **1000**, as we'd like to allow the solver enough iterations to find a good solution:

```
lr = LogisticRegression(solver='saga', penalty='l1',
                        max_iter=1000)
```

5. Import the **Pipeline** class and create a pipeline with the scaler and the logistic regression model, using the names '**scaler**' and '**model**' for the steps, respectively:

```
from sklearn.pipeline import Pipeline
scale_lr_pipeline = Pipeline(
    steps=[('scaler', min_max_sc), ('model', lr)])
```

6. Use the **get_params** and **set_params** methods to see how to view the parameters from each stage of the pipeline and change them (execute each of the following lines in a separate cell in your notebook and observe the output):

```
scale_lr_pipeline.get_params()
scale_lr_pipeline.get_params()['model__C']
scale_lr_pipeline.set_params(model__C = 2)
```

7. Create a smaller range of C values to test with cross-validation, as these models will take longer to train and test with more data than our previous exercise; we recommend $C = [10^2, 10, 1, 10^{-1}, 10^{-2}, 10^{-3}]$:

```
C_val_exponents = np.linspace(2, -3, 6)
C_vals = np.float(10)**C_val_exponents
```

8. Make a new version of the **cross_val_C_search** function, called **cross_val_C_search_pipe**. Instead of the **model** argument, this function will take a **pipeline** argument. The changes inside the function will be to set the C value using **set_params(model__C = <value you want to test>)** on the pipeline, replacing the model with the pipeline for the **fit** and **predict_proba** methods, and accessing the C value using **pipeline.get_params()['model__C']** for the printed status update.

The changes are as follows:

```
def cross_val_C_search_pipe(k_folds, C_vals,
                             pipeline, X, Y):
    ##[...]
    pipeline.set_params(model__C = C_vals[c_val_counter])
    ##[...]
    pipeline.fit(X_cv_train, y_cv_train)
    ##[...]
    y_cv_train_predict_proba =
    pipeline.predict_proba(X_cv_train)
    ##[...]
    y_cv_test_predict_proba =
    pipeline.predict_proba(X_cv_test)
    ##[...]
    print('Done with C =
    {}'.format(pipeline.get_params()\
                [ 'model__C' ]))
```

Note

For the complete code, refer to <https://packt.link/AsQmK>.

9. Run this function as in the previous exercise, but using the new range of C values, the pipeline you created, and the features and response variable from the training split of the case study data. You may see warnings here, or in later steps, regarding the non-convergence of the solver; you could experiment with the `tol` or `max_iter` options to try and achieve convergence, although the results you obtain with `max_iter = 1000` are likely to be sufficient. Here is the code to do this:

```
cv_train_roc_auc, cv_test_roc_auc, cv_test_roc = \
cross_val_C_search_pipe(k_folds, C_vals,
scale_lr_pipeline,
                        X_train, y_train)
```

You will obtain the following output:

```
Done with C = 100.0
Done with C = 10.0
Done with C = 1.0
Done with C = 0.1
Done with C = 0.01
Done with C = 0.001
```

10. Plot the average training and test ROC AUC across folds, for each C value, using the following code:

```
plt.plot(C_val_exponents, np.mean(cv_train_roc_auc,
axis=0),
         '-o', label='Average training score')
plt.plot(C_val_exponents, np.mean(cv_test_roc_auc,
axis=0),
         '-x', label='Average testing score')
plt.ylabel('ROC AUC')
plt.xlabel('log$_{10}$$(C)$')
plt.legend()
plt.title('Cross-validation on Case Study problem')
```

You will obtain the following output:

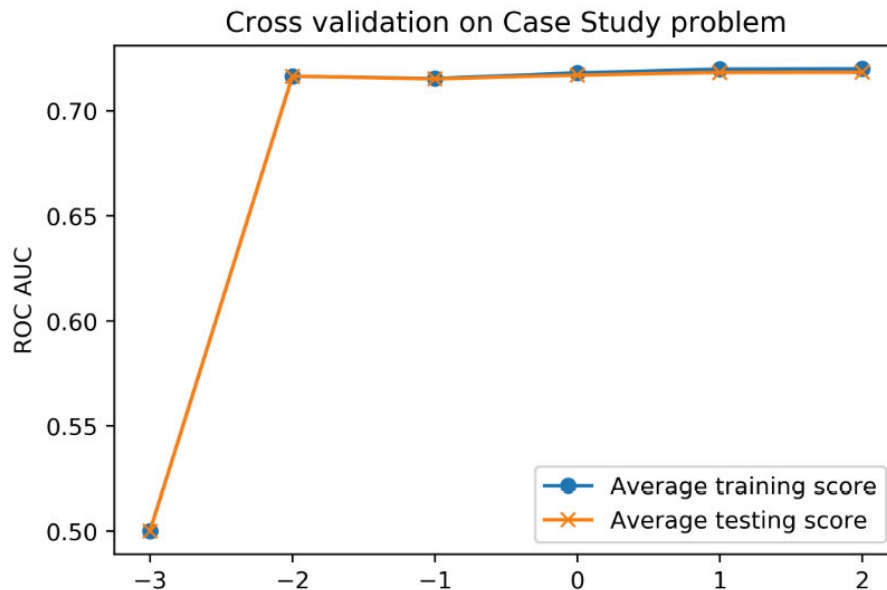


Figure 4.25: Cross-validation test performance

You should notice that regularization does not impart much benefit here, as may be expected: for lower C values, which correspond to stronger regularization, model testing (as well as training) performance decreases. While we are able to increase model performance over our previous efforts by using all the features available, it appears there is no overfitting going on. Instead, the training and test scores are about the same. Instead of overfitting, it's possible that we may be underfitting. Let's try engineering some interaction features to see if they can improve performance.

11. Create interaction features for the case study data and confirm that the number of new features makes sense using the following code:

```
from sklearn.preprocessing import PolynomialFeatures
make_interactions = PolynomialFeatures(degree=2,
                                       interaction_on
                                       ly=True,
                                       include_bias=F
                                       else)
X_interact = make_interactions.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_interact, df['default payment next
    month'].values,
    test_size=0.2, random_state=24)
print(X_train.shape)
```

```
print(X_test.shape)
```

You will obtain the following output:

```
(21331, 153)
(5333, 153)
```

From this you should see the new number of features is 153, which is $17 + "17 \text{ choose } 2" = 17 + 136 = 153$. The "*17 choose 2*" part comes from choosing all possible combinations of 2 features to interact from the 17 original features.

12. Repeat the cross-validation procedure and observe the model performance when using interaction features; that is, repeat *steps 9* and *10*. Note that this will take substantially more time, due to the larger number of features, but it will probably take less than 10 minutes.

You will obtain the following output:

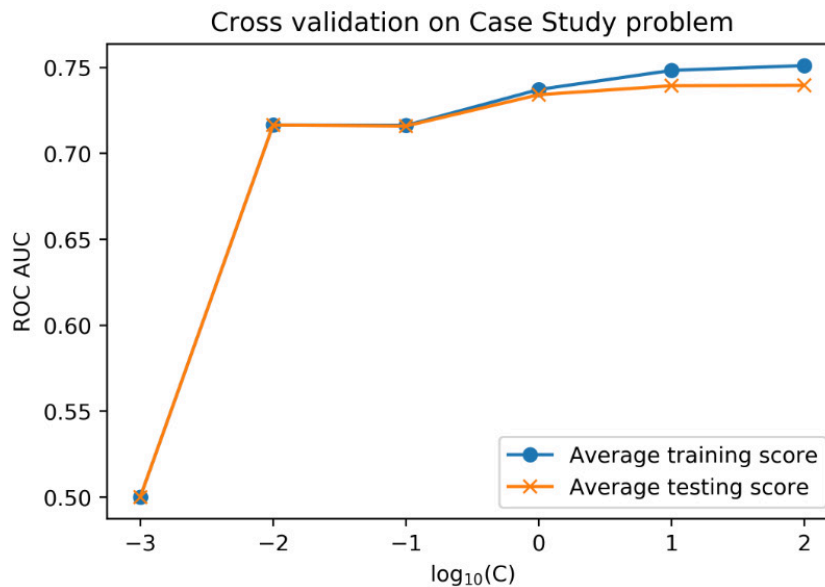


Figure 4.26: Improved cross-validation test performance from adding interaction features

So, does the average cross-validation test performance improve with the interaction features? Is regularization useful?

Engineering the interaction features increases the best model test score to about $ROC AUC = 0.74$ on average across the folds, from about 0.72 without including interactions. These scores happen at $C = 100$,

that is, with negligible regularization. On the plot of training versus test scores for the model with interactions, you can see that the training score is a bit higher than the test score, so it could be said that some amount of overfitting is going on. However, we cannot increase the test score through regularization here, so this may not be a problematic instance of overfitting. In most cases, whatever strategy yields the highest test score is the best strategy.

In summary, adding interaction features improved cross-validation performance, and regularization appears not to be useful for the case study at this point, using a logistic regression model. We will reserve the step of fitting on all the training data for later when we've tried other models in cross-validation to find the best model.

5. Decision Trees and Random Forests

Activity 5.01: Cross-Validation Grid Search with Random Forest

Solution:

1. Create a dictionary representing the grid for the **max_depth** and **n_estimators** hyperparameters that will be searched. Include depths of 3, 6, 9, and 12, and 10, 50, 100, and 200 trees. Leave the other hyperparameters at their defaults. Create the dictionary using this code:

```
rf_params = {'max_depth':[3, 6, 9, 12],  
             'n_estimators':[10, 50, 100, 200]}
```

Note

*There are many other possible hyperparameters to search over. In particular, the scikit-learn documentation for random forest indicates that "The main parameters to adjust when using these methods are **n_estimators** and **max_features**" and that "Empirical good de-*

fault values are ... `max_features=sqrt(n_features)` for classification tasks."

Source: [https://scikit-](https://scikit-learn.org/stable/modules/ensemble.html#parameters)

learn.org/stable/modules/ensemble.html#parameters

*For the purposes of this book, we will use `max_features='auto'` (which is equal to `sqrt(n_features)`) and limit our exploration to `max_depth` and `n_estimators` for the sake of a shorter runtime. In a real-world situation, you should explore other hyperparameters according to how much computational time you can afford. Remember that in order to search in especially large parameter spaces, you can use **RandomizedSearchCV** to avoid exhaustively calculating metrics for every combination of hyperparameters in the grid.*

2. Instantiate a **GridSearchCV** object using the same options that we have previously used in this chapter, but with the dictionary of hyperparameters created in step 1 here. Set **verbose=2** to see the output for each fit performed. You can reuse the same random forest model object, **rf**, that we have been using or create a new one. Create a new random forest object and instantiate the **GridSearchCV** class using this code:

```
rf = RandomForestClassifier(n_estimators=10,\
                           criterion='gini',\
                           max_depth=3,\
                           min_samples_split=2,\
                           min_samples_leaf=1,\
                           min_weight_fraction_leaf=\
0.0,\
                           max_features='auto',\
                           max_leaf_nodes=None,\
                           min_impurity_decrease=0.0,\
                           min_impurity_split=None,\
                           bootstrap=True,\
                           oob_score=False,\
                           n_jobs=None,
```

```

        random_state=4,\
        verbose=0,\
        warm_start=False,\
        class_weight=None)
cv_rf = GridSearchCV(rf, param_grid=rf_params,\
                    scoring='roc_auc',\
                    n_jobs=-1,\
                    refit=True,\
                    cv=4,\
                    verbose=2,\
                    error_score=np.nan,\
                    return_train_score=True)

```

3. Fit the **GridSearchCV** object on the training data. Perform the grid search using this code:

```
cv_rf.fit(X_train, y_train)
```

Because we chose the **verbose=2** option, you will see a relatively large amount of output in the notebook. There will be output for each combination of hyperparameters and, for each fold, as it is fitted and tested. Here are the first few lines of output:

```

Fitting 4 folds for each of 16 candidates, totalling 64 fits
[CV] max_depth=3, n_estimators=10 .....
[CV] ..... max_depth=3, n_estimators=10, total= 0.1s
[CV] max_depth=3, n_estimators=10 .....
[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.
[Parallel(n_jobs=1)]: Done 1 out of 1 | elapsed: 0.1s remaining: 0.0s

[CV] ..... max_depth=3, n_estimators=10, total= 0.1s
[CV] max_depth=3, n_estimators=10 .....
[CV] ..... max_depth=3, n_estimators=10, total= 0.1s
[CV] max_depth=3, n_estimators=10 .....
[CV] ..... max_depth=3, n_estimators=10, total= 0.1s
[CV] max_depth=3, n_estimators=50 .....
[CV] ..... max_depth=3, n_estimators=50, total= 0.5s

```

Figure 5.22: The verbose output from cross-validation

While it's not necessary to see all this output for shorter cross-validation procedures, for longer ones, it can be reassuring to see that the cross-validation is working and to give you an idea of how long the fits are taking for various combinations of hyperparameters. If things are taking too long, you may want to interrupt the kernel by pushing the stop button (square) at the top of the notebook and choosing hyperparameters that will take less time to run, or use a more limited set of hyperparameters.

When this is all done, you should see the following output:

```
[Parallel(n_jobs=1)]: Done 64 out of 64 | elapsed: 2.3min finished
GridSearchCV(cv=4,
              estimator=RandomForestClassifier(max_depth=3, n_estimators=10,
                                                random_state=4),
              n_jobs=1,
              param_grid={'max_depth': [3, 6, 9, 12],
                           'n_estimators': [10, 50, 100, 200]},
              return_train_score=True, scoring='roc_auc', verbose=2)
```

Figure 5.23: The cross-validation output upon completion

This cross-validation job took about 2 minutes to run. As your jobs grow, you may wish to explore parallel processing with the **n_jobs** parameter to see whether it's possible to speed up the search. Using **n_jobs=-1** for parallel processing, you should be able to achieve shorter runtimes than with serial processing. However, with parallel processing, you won't be able to see the output of each individual model fitting operation, as shown in *Figure 5.23*.

4. Put the results of the grid search in a pandas DataFrame. Use this code to put the results in a DataFrame:

```
cv_rf_results_df = pd.DataFrame(cv_rf.cv_results_)
```

5. Create a **pcolormesh** visualization of the mean testing score for each combination of hyperparameters. Here is the code to create a mesh graph of cross-validation results. It's similar to the example graph that we created previously, but with annotation that is specific to the cross-validation we performed here:

```
ax_rf = plt.axes()
pcolor_graph = ax_rf.pcolormesh\
                (xx_rf, yy_rf,\
                 cv_rf_results_df['mean_test_score']\
                 .values.reshape((4,4)), cmap=cm_rf)
plt.colorbar(pcolor_graph, label='Average testing ROC AUC')
ax_rf.set_aspect('equal')
ax_rf.set_xticks([0.5, 1.5, 2.5, 3.5])
ax_rf.set_yticks([0.5, 1.5, 2.5, 3.5])
ax_rf.set_xticklabels\
([str(tick_label) for tick_label in
 rf_params['n_estimators']])
```

```
ax_rf.set_yticklabels\
([str(tick_label) for tick_label in
 rf_params['max_depth']])
ax_rf.set_xlabel('Number of trees')
ax_rf.set_ylabel('Maximum depth')
```

The main change from our previous example is that instead of plotting the integers from 1 to 16, we're plotting the mean testing scores that we've retrieved and reshaped with

`cv_rf_results_df['mean_test_score'].values.reshape((4,4)).`

The other new things here are that we are using list comprehensions to create lists of strings for tick labels, based on the numerical values of hyperparameters in the grid. We access them from the dictionary that we defined, and then convert them individually to the **str** (string) data type within the list comprehension, for example, **`ax_rf.set_xticklabels([str(tick_label) for tick_label in rf_params['n_estimators']])`**. We have already set the tick locations to the places where we want the ticks using **`set_xticks`**.

Also, we make a square-shaped graph using

`ax_rf.set_aspect('equal')`. The graph should appear as follows:

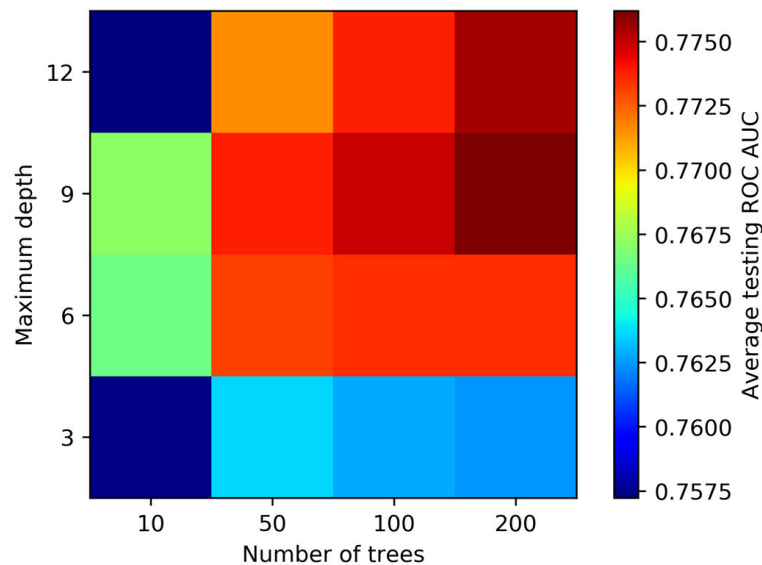


Figure 5.24: Results of cross-validation of a random forest over a grid with two hyperparameters

6. Conclude which set of hyperparameters to use.

What can we conclude from our grid search? There certainly seems to be an advantage to using trees with a depth of more than 3. Of

the parameter combinations that we tried, `max_depth=9` with 200 trees yields the best average testing score, which you can look up in the DataFrame and confirm is ROC AUC = 0.776.

This is the best model we've found from all of our efforts so far.

In a real-world scenario, we'd likely do a more thorough search.

Some good next steps would be to try a larger number of trees and not spend any more time with `n_estimators < 200`, since we know that we need at least 200 trees to get the best performance. You may search a more granular space of `max_depth` instead of jumping by 3s, as we've done here, and try a couple of other hyperparameters, such as `max_features`. For our purposes, however, we will assume that we've found the optimal hyperparameters here and move forward..

6. Gradient Boosting, XGBoost, and SHAP Values

Activity 6.01: Modeling the Case Study Data with XGBoost and Explaining the Model with SHAP

Solution:

In this activity, we'll take what we've learned in this chapter with a synthetic dataset and apply it to the case study data. We'll see how an XGBoost model performs on a validation set and explain the model predictions using SHAP values. We have prepared the dataset for this activity by replacing the samples that had missing values for the `PAY_1` feature, that we had previously ignored, while maintaining the same train/test split for the samples with no missing values. You can see how the data was prepared in the Appendix to the notebook for this activity.

1. Load the case study data that has been prepared for this exercise. The file path is `../Data/Activity_6_01_data.pkl` and the variables are: `features_response`, `X_train_all`, `y_train_all`, `X_test_all`, `y_test_all`:

```
with open('../Data/Activity_6_01_data.pkl', 'rb')
as f:
    features_response, X_train_all, y_train_all,
    X_test_all,\
    y_test_all = pickle.load(f)
```

2. Define a validation set to train XGBoost with early stopping:

```
from sklearn.model_selection import train_test_split
X_train_2, X_val_2, y_train_2, y_val_2 = \
train_test_split(X_train_all, y_train_all,\
                 test_size=0.2, random_state=24)
```

3. Instantiate an XGBoost model. We'll use the **lossguide** grow policy and examine validation set performance for several values of **max_leaves**:

```
xgb_model_4 = xgb.XGBClassifier(
    n_estimators=1000,
    max_depth=0,
    learning_rate=0.1,
    verbosity=1,
    objective='binary:logistic',
    use_label_encoder=False,
    n_jobs=-1,
    tree_method='hist',
    grow_policy='lossguide')
```

4. Search values of **max_leaves** from 5 to 200, counting by 5's:

```
max_leaves_values = list(range(5,205,5))
```

5. Create the evaluation set for early stopping:

```
eval_set_2 = [(X_train_2, y_train_2), (X_val_2,
y_val_2)]
```

6. Loop through hyperparameter values and create a list of validation ROC AUCs, using the same technique as in *Exercise 6.01*:

Randomized Grid Search for Tuning XGBoost Hyperparameters:

```

%%time
val_aucs = []
for max_leaves in max_leaves_values:
    #Set parameter and fit model
    xgb_model_4.set_params(**
{'max_leaves':max_leaves})
    xgb_model_4.fit(X_train_2, y_train_2,\
                    eval_set=eval_set_2,\
                    eval_metric='auc',\
                    verbose=False,\
                    early_stopping_rounds=30)

    #Get validation score
    val_set_pred_proba =
xgb_model_4.predict_proba(X_val_2)[: ,1]
    val_aucs.append(roc_auc_score(y_val_2,
val_set_pred_proba))

```

7. Create a data frame of the hyperparameter search results and plot the validation AUC against **max_leaves**:

```

max_leaves_df_2 = \
pd.DataFrame({'Max leaves':max_leaves_values,\
              'Validation AUC':val_aucs})

mpl.rcParams['figure.dpi'] = 400
max_leaves_df_2.set_index('Max leaves').plot()

```

The plot should look something like this:

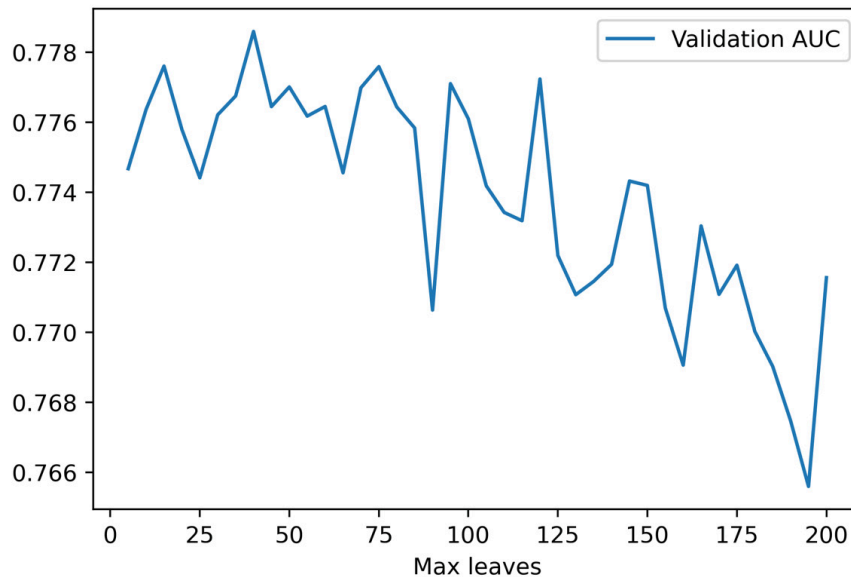


Figure 6.15: Validation AUC versus **max_leaves** for the case study data

Although the relationship is somewhat noisy, we see that in general, lower values of **max_leaves** result in a higher validation set ROC AUC. This is because limiting the complexity of trees by allowing fewer leaves results in less overfitting, and increases the validation set score.

8. Observe the number of **max_leaves** corresponding to the highest ROC AUC on the validation set:

```
max_auc_2 = max_leaves_df_2['Validation AUC'].max()
max_auc_2
max_ix_2 = max_leaves_df_2['Validation AUC'] ==
max_auc_2
max_leaves_df_2[max_ix_2]
```

The result should be as follows:

Max leaves	Validation AUC
7	0.778592

Figure 6.16: Optimal **max_leaves** and validation set AUC for the case study data

We would like to interpret these results in light of our previous efforts in modeling the case study data. This is not a perfect comparison, because here we have missing values in the training and validation data, while previously we ignored them, and here we only

have one validation set, as opposed to the k-folds cross-validation used earlier (although the interested reader could try using k-folds cross-validation for multiple training/validation splits in XGBoost with early stopping).

However, even given these limitations, the validation results here should provide a measure of out-of-sample performance similar to the k-folds cross-validation we performed earlier. We note that the validation ROC AUC here of 0.779 here is a bit higher than the 0.776 obtained previously with random forest in *Activity 5.01, Cross-Validation Grid Search with Random Forest*, from *Chapter 5, Decision Trees and Random Forests*. These validation scores are fairly similar and it would probably be fine to use either model in practice. We'll now move forward with the XGBoost model.

9. Refit the XGBoost model with the optimal hyperparameter:

```
xgb_model_4.set_params(**{'max_leaves':40})
xgb_model_4.fit(X_train_2, y_train_2,
                eval_set=eval_set_2,
                    eval_metric='auc',
                    verbose=False,
                early_stopping_rounds=30)
```

10. So that we can examine SHAP values for the validation set, make a data frame of this data:

```
X_val_2_df = pd.DataFrame(data=X_val_2,
                           columns=features_response[:
-1])
```

11. Create an SHAP explainer for our new model using the validation data as the background dataset, obtain the SHAP values, and make a summary plot:

```
explainer_2 = shap.explainers.Tree(xgb_model_4,
data=X_val_2_df)
shap_values_2 = explainer_2(X_val_2_df)
mpl.rcParams['figure.dpi'] = 75
shap.summary_plot(shap_values_2.values, X_val_2_df)
```

The plot should look like this:

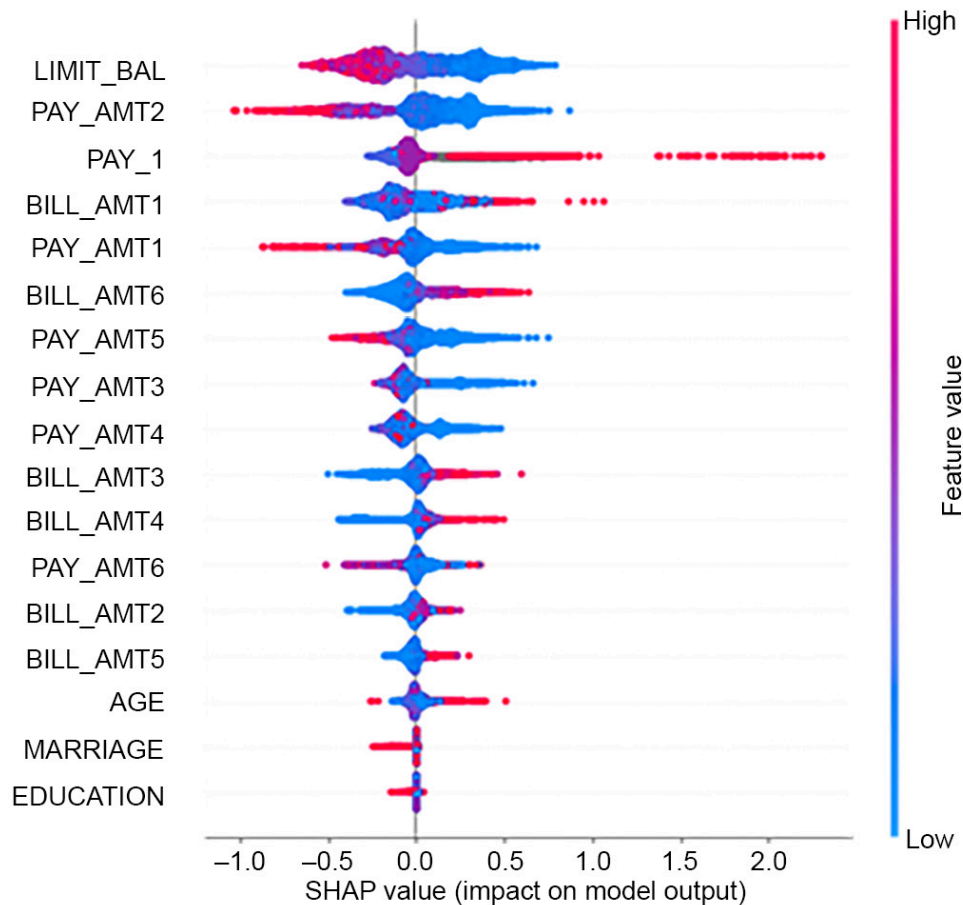


Figure 6.17: SHAP values for the XGBoost model of the case study data on the validation set

From *Figure 6.17*, we can see that the most important features in the XGBoost model are somewhat different from those in the random forest model we explored in *Chapter 5, Decision Trees and Random Forests (Figure 5.15)*. No longer is **PAY_1** the most important feature, although it is still quite important at number 3.

LIMIT_BAL, the borrower's credit limit, is now the most important feature. This makes sense as an important feature as the lender has likely based the credit limit on how risky a borrower is, so it should be a good predictor of the risk of default.

Let's explore whether **LIMIT_BAL** has any interesting SHAP interactions with other features. Instead of specifying which feature to color the scatter plot by, we can let the **shap** package pick the feature that has the most interaction by not indexing the explainer object for the color argument.

12. Make a scatter plot of **LIMIT_BAL** SHAP values, colored by the feature with the strongest interaction:

```
shap.plots.scatter(shap_values_2[:, 'LIMIT_BAL'],
                   color=shap_values_2)
```

The plot should look like this:

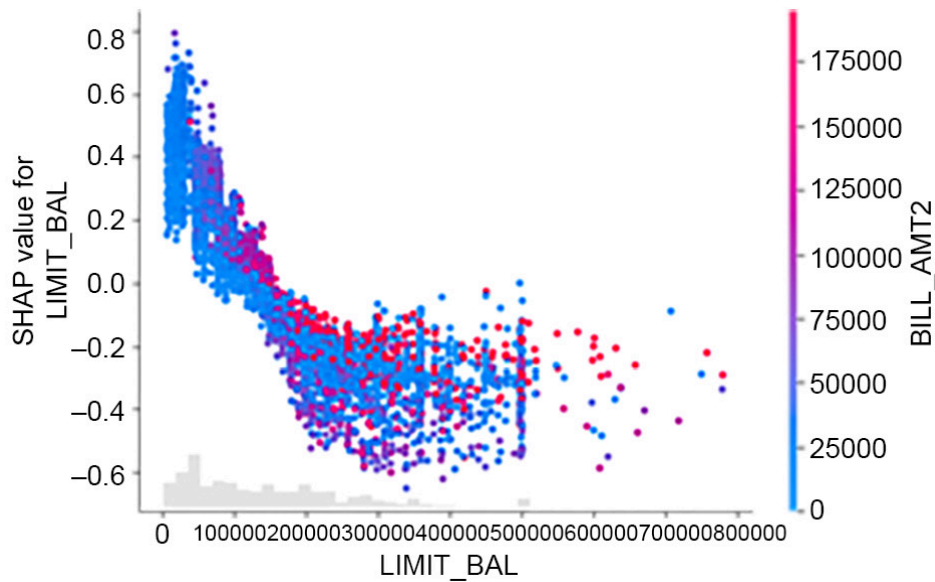


Figure 6.18: Scatter plot of SHAP values of **LIMIT_BAL** and the feature with the strongest interaction

BILL_AMT2, the amount of the bill from two months previous, has the strongest interaction with **LIMIT_BAL**. We can see that for most values of **LIMIT_BAL**, if the bill was particularly high, this leads to more positive SHAP values, meaning an increased risk of default. This can be observed by noting that most of the reddest colored dots appear along the top of the band of dots in *Figure 6.18*. This makes intuitive sense: even if a borrower was given a large credit limit, if their bill becomes very large, this may signal an increased risk of default.

Finally, we will save the model along with the training and test data for analysis and delivery to our business partner. We accomplish this using Python's **pickle** functionality.

13. Save the trained model along with the training and test data to a file:

```
with open('../Data/xgb_model_w_data.pkl', 'wb') as f:
    pickle.dump([X_train_all, y_train_all,\
                  X_test_all, y_test_all,\
                  xgb_model_4], f)
```

7. Test Set Analysis, Financial Insights, and Delivery to the Client

Activity 7.01: Deriving Financial Insights

Solution:

1. Using the testing set, calculate the cost of all defaults if there were no counseling program.

Use this code for the calculation:

```
cost_of_defaults = np.sum(y_test_all *  
X_test_all[:,5])  
cost_of_defaults
```

The output should be this:

```
60587763.0
```

2. Calculate by what percent the cost of defaults can be decreased by the counseling program.

The potential decrease in the cost of default is the greatest possible net savings of the counseling program, divided by the cost of all defaults in the absence of a program:

```
net_savings[max_savings_ix]/cost_of_defaults
```

The output should be this:

```
0.2214260658542551
```

Results indicate that we can decrease the cost of defaults by 22% using a counseling program, guided by predictive modeling.

3. Calculate the net savings per account (considering all accounts it might be possible to counsel, in other words relative to the whole test set) at the optimal threshold.

Use this code for the calculation:

```
net_savings[max_savings_ix]/len(y_test_all)
```

The output should be as follows:

```
2259.2977433479286
```

Results like these help the client scale the potential amount of savings they could create with the counseling program, to as many accounts as they serve.

4. Plot the net savings per account against the cost of counseling per account for each threshold.

Create the plot with this code:

```
plt.plot(total_cost/len(y_test_all),
         net_savings/len(y_test_all))
plt.xlabel\
('Upfront investment: cost of counselings per account (NT$)')
plt.ylabel('Net savings per account (NT$)')
```

The resulting plot should appear like this:

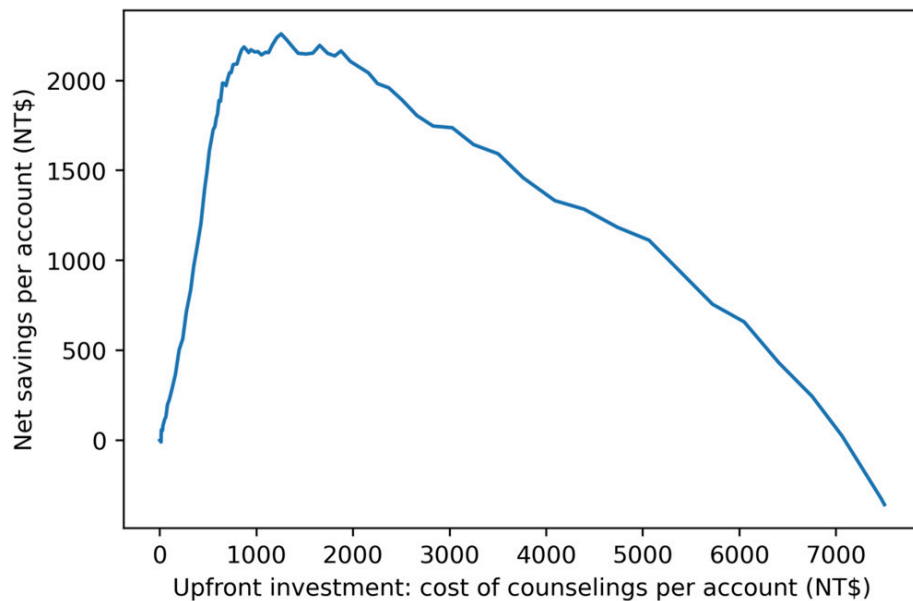


Figure 7.14: The initial cost of the counseling program needed to achieve a given amount of savings

This indicates how much money the client needs to budget to the counseling program in a given month, to achieve a given amount of savings. It looks like the greatest benefit can be created by budgeting up to about NT\$1300 per account (you could find the exact budgeted amount corresponding to maximum net savings using `np.argmax`). However, net savings are relatively flat for upfront investments between NT\$1000 and 2000, being lower outside that range. The client may not actually be able to budget this much for

the program. However, this graphic gives them evidence to argue for a larger budget if they need to.

This result corresponds to our graphic from the previous exercise. Although we've shown the optimal threshold is 0.36, it may be fine for the client to use a higher threshold up to about 0.5, thus making fewer positive predictions, offering counseling to fewer account holders, and having a smaller upfront program cost. *Figure 7.14* shows how this plays out in terms of cost and net savings per account.

5. Plot the fraction of accounts predicted as positive (this is called the "flag rate") at each threshold.

Use this code to plot the flag rate against the threshold:

```
plt.plot(thresholds, n_pos_pred/len(y_test_all))
plt.ylabel('Flag rate')
plt.xlabel('Threshold')
```

The plot should appear as follows:

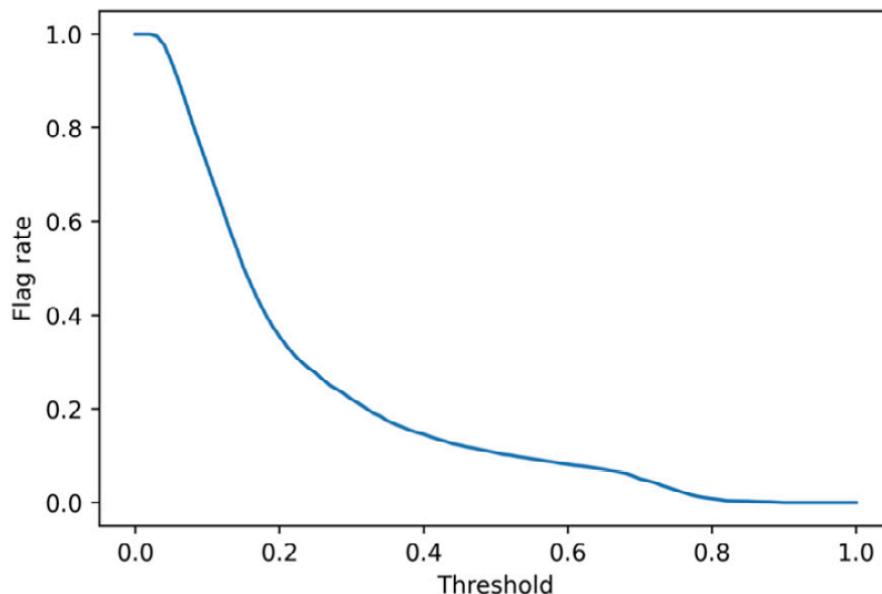


Figure 7.15: Flag rate against threshold for the credit counseling program

This plot shows the fraction of people who will be predicted to default and therefore will be recommended outreach at each threshold. It appears that at the optimal threshold of 0.36, only about 20% of accounts will be flagged for counseling. This shows how using a model to prioritize accounts for counseling can help focus on the

right accounts and reduce wasted resources. Higher thresholds, which may result in nearly optimal savings up to a threshold of about 0.5 as shown in *Figure 7.12* (*Chapter 7, Test Set Analysis, Financial Insights, and Delivery to the Client*) result in lower flag rates.

6. Plot a precision-recall curve for the testing data using the following code:

```
plt.plot(n_true_pos/sum(y_test_all),\
         np.divide(n_true_pos, n_pos_pred))
plt.xlabel('Recall')
plt.ylabel('Precision')
```

The plot should look like this:

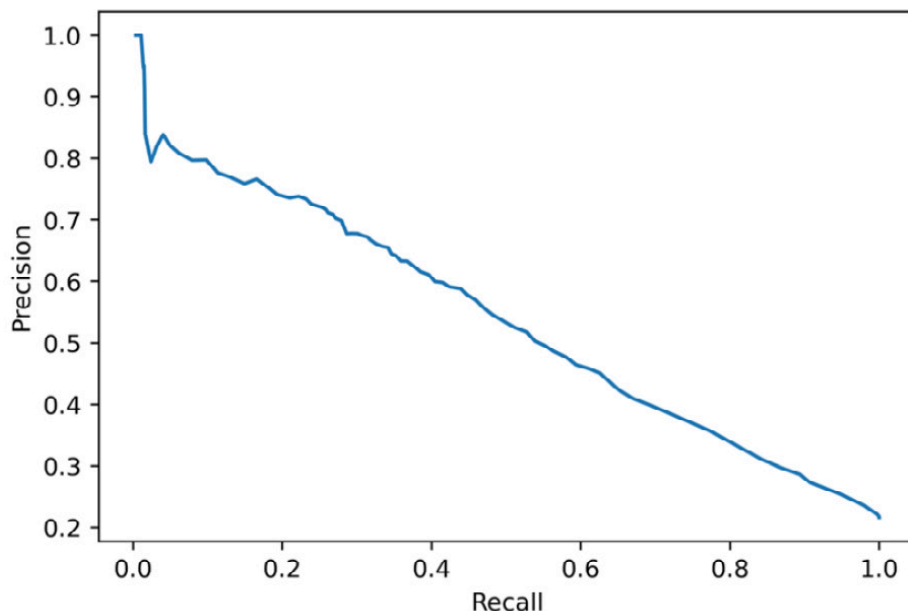


Figure 7.16: Precision-recall curve

Figure 7.16 shows that in order to start getting a true positive rate (that is, recall) much above 0, we need to accept a precision of about 0.8 or lower.

Precision and recall have a direct link to the cost and savings of the program: the more precise our predictions are, the less money we are wasting on counseling due to incorrect model predictions. And, the higher the recall, the more savings we can create by successfully identifying accounts that would default. Compare the code in this step to the code used to calculate costs and savings in the previous exercise to see this.

To see the connection of precision and recall with the threshold used to define positive and negative predictions, it can be instructive to plot them separately.

7. Plot precision and recall separately on the y-axis against threshold on the x-axis.

Use this code to produce the plot:

```
plt.plot(thresholds, np.divide(n_true_pos,
                              n_pos_pred),
         label='Precision')
plt.plot(thresholds, n_true_pos/sum(y_test_all),
         label='Recall')
plt.xlabel('Threshold')
plt.legend()
```

The plot should appear as follows:

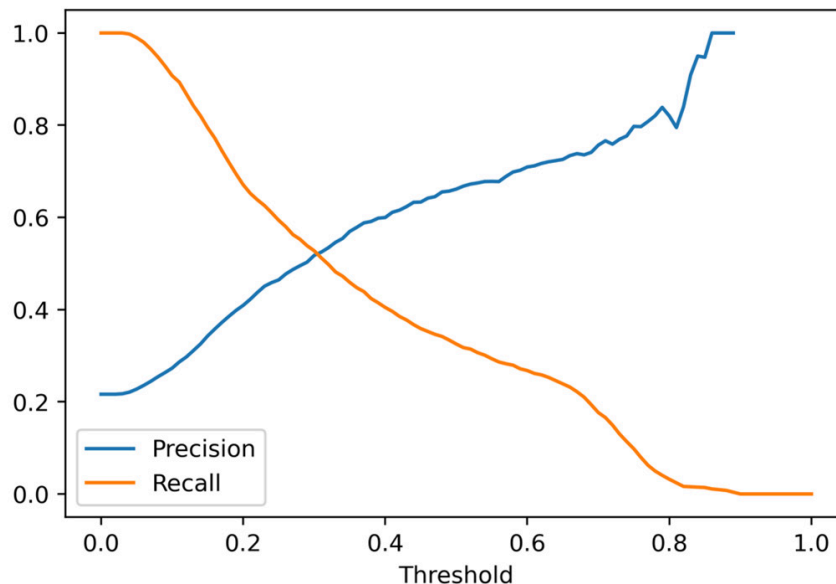


Figure 7.17: Precision and recall plotted separately against the threshold

This plot sheds some light on why the optimal threshold turned out to be 0.36. While the optimal threshold also depends on the financial analysis of costs and savings, we can see here that the steepest part of the initial increase in precision, which represents the correctness of positive predictions and is therefore a measure of how cost-effective

the model-guided counseling can be, happens up to a threshold of about 0.36.